

汽车理论课程设计

专业名称：车辆工程

班 级：2023370501

学 号：2023901021

学生姓名：李思成

2026 年 6 月 1 日

目 录

题目一：轻型货车动力性能计算（习题 1.3）	3
1.1 题目描述	3
1.2 已知参数	3
1.3 变速器传动比（五档）	3
1.4 计算公式	3
1.5 计算结果	4
题目二：轻型货车燃油经济性计算（习题 2.7）	6
2.1 题目描述	6
2.2 负荷特性拟合系数	6
2.3 计算公式	6
2.4 计算结果	7
题目三：中型货车制动性能计算（习题 4.3）	9
3.1 题目描述	9
3.2 计算公式	9
3.3 计算结果	9
附录：编程程序代码	11
A.1 题目一：动力性能计算程序	11
A.2 题目二：燃油经济性计算程序	18
A.3 题目三：制动性能计算程序	25

题目一：轻型货车动力性能计算（习题 1.3）

1.1 题目描述

确定一轻型货车的动力性能（货车装用五档变速器）：

- (1) 绘制汽车驱动力与行驶阻力平衡图。
- (2) 求汽车最高车速、最大爬坡度及克服该坡度时相应的附着率。
- (3) 绘制汽车行驶加速度倒数曲线，用计算机求汽车用 2 档起步加速行驶至 70km/h 的加速时间。

1.2 已知参数

参数	数值
装载质量	2000 kg
整车整备质量	1800 kg
总质量	3880 kg
车轮半径	0.367 m
传动系机械效率 η_t	0.85
滚动阻力系数 f	0.013
空气阻力系数×迎风面积 CDA	2.77 m ²
主减速器传动比 i_0	5.83
飞轮转动惯量 I_f	0.218 kg·m ²
二前轮转动惯量 I_{w1}	1.798 kg·m ²
四后轮转动惯量 I_{w2}	3.598 kg·m ²
轴距 L	3.2 m
质心至前轴距离（满载）a	1.947 m
质心高（满载）h _g	0.9 m

1.3 变速器传动比（五档）

档位	1 档	2 档	3 档	4 档	5 档
传动比 i_g	5.56	2.769	1.644	1.00	0.793

1.4 计算公式

- (1) 发动机外特性转矩拟合公式（ T_q - n 曲线）：

$$T_q = -19.313 + 295.27(n/1000) - 165.44(n/1000)^2 + 40.874(n/1000)^3 - 3.8445(n/1000)^4$$

- (2) 各档车速：

$$u_a = 0.377 \times r \times n / (i_g \times i_0) \text{ [km/h]}$$

- (3) 各档驱动力：

$$F_t = T_q \times i_g \times i_0 \times \eta_t / r \text{ [N]}$$

(4) 行驶阻力:

$$F_f + F_w = G \times f + CDA \times u_a^2 / 21.15 \text{ [N]}$$

(5) 旋转质量换算系数:

$$\delta = 1 + (\sum Iw + I_f \times i_g^2 \times i_0^2 \times \eta_t) / (m \times r^2)$$

(6) 加速度:

$$a = (F_t - F_f - F_w) / (\delta \times m) \text{ [m/s}^2\text{]}$$

(7) 加速时间（图解积分法）:

$$t = \int (1/a) du = (1/3.6) \times \int (1/a) du \text{ [s]}$$

1.5 计算结果

(1) 驱动力-行驶阻力平衡图如图 1 所示。

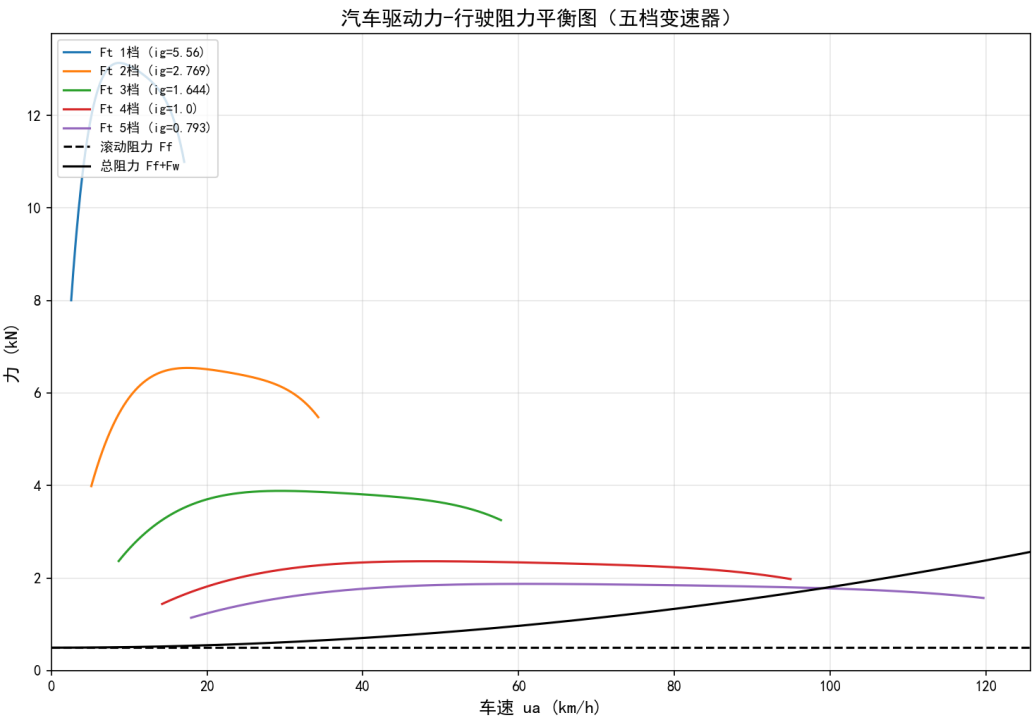


图 1 汽车驱动力-行驶阻力平衡图

(2) 最高车速与最大爬坡度:

项目	结果
最高车速 u_{a_max}	99.08 km/h
发动机最大扭矩 T_{q_max}	174.97 N·m @ 2044 r/min
发动机最大功率 P_{e_max}	61.73 kW @ 3864 r/min
最大爬坡度 i	35.02% (19.30°)
最大爬坡度对应车速	10.01 km/h
最大爬坡度时附着率 $C\phi_2$	0.4348

(3) 加速度倒数曲线与加速时间：

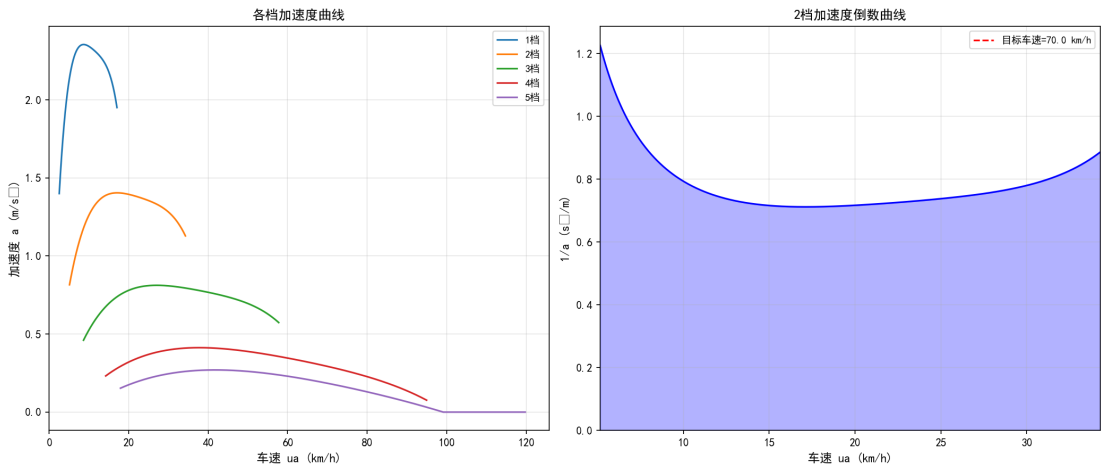


图 2 各档加速度曲线与 2 档加速度倒数曲线

项目	时间
2 档加速段时间	6.34 s
换挡时间	0.80 s
3 档加速段时间	9.15 s
2 档起步加速至 70km/h 总时间	16.29 s

题目二：轻型货车燃油经济性计算（习题 2.7）

2.1 题目描述

货车的参数与题目一相同。负荷特性曲线的拟合公式为：

$$b = B_0 + B_1 \cdot P_e + B_2 \cdot P_e^2 + B_3 \cdot P_e^3 + B_4 \cdot P_e^4$$

式中， b 为燃油消耗率 $[g/(kW \cdot h)]$ ， P_e 为发动机净功率(kW)。

- (1) 绘制该轻型货车的功率平衡图。
- (2) 绘制最高挡与次高挡的等速百公里油耗曲线。
- (3) 计算六工况循环行驶的百公里油耗。

2.2 负荷特性拟合系数

n (r/min)	B0	B1	B2	B3	B4
815	1326.8	-416.46	72.379	-5.8629	0.17768
1207	1354.7	-303.98	36.657	-2.0553	0.043072
1614	1284.4	-189.75	14.524	-0.51184	0.0068164
2012	1122.9	-121.59	7.0035	-0.18517	0.0018555
2603	1141.0	-98.893	4.4763	-0.091077	0.00068906
3006	1051.2	-73.714	2.8593	-0.05138	0.00035032
3403	1233.9	-84.478	2.9788	-0.047449	0.00028230
3804	1129.7	-45.291	0.71113	-0.00075215	- 0.000038568

2.3 计算公式

- (1) 发动机功率：

$$P_e = T_q \times n / 9550 \text{ [kW]}$$

- (2) 阻力功率：

$$P_f = (F_f + F_w) \times u_a / (3600 \times \eta_t) \text{ [kW]}$$

- (3) 等速百公里油耗：

$$Q_s = P_e \times b / (1.02 \times u_a \times \gamma) \text{ [L/100km]}$$

- (4) 瞬时燃油消耗量：

$$Q_t = P_e \times b / (3600 \times \rho) \text{ [mL/s]}$$

式中， γ 为燃油重度，93#汽油 $\gamma = \rho g = 0.715 \times 9.8 = 7.007 \text{ N/L}$ ； $\rho = 0.715 \text{ kg/L}$ 。

2.4 计算结果

(1) 功率平衡图如图 3 所示。

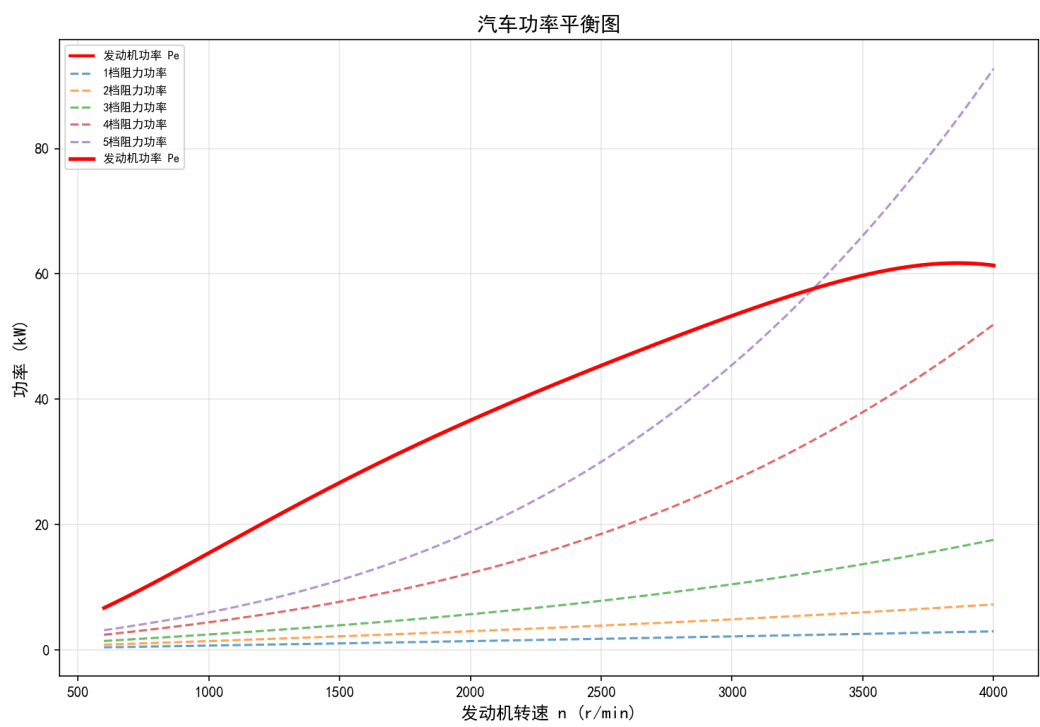


图 3 汽车功率平衡图

(2) 等速百公里油耗曲线如图 4 所示。

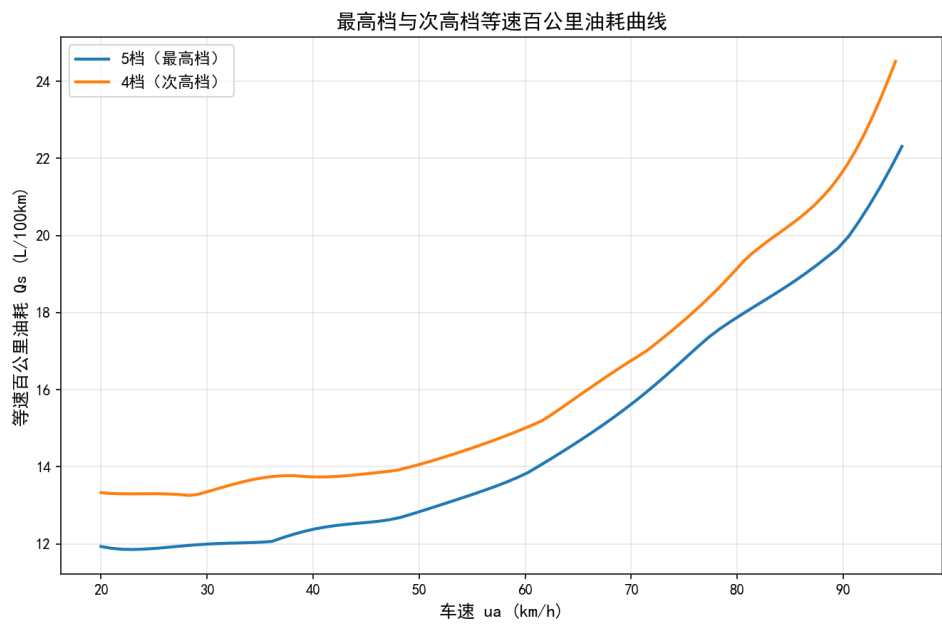


图 4 最高档与次高档等速百公里油耗曲线

(3) 等速百公里油耗特征值：

项目	油耗
5 档（最高档）最低油耗	11.99 L/100km @ 30 km/h
5 档 60km/h 等速油耗	13.84 L/100km
5 档 80km/h 等速油耗	17.96 L/100km
4 档（次高档）最低油耗	13.35 L/100km @ 30 km/h
4 档 60km/h 等速油耗	15.04 L/100km
4 档 80km/h 等速油耗	19.22 L/100km

(4) 六工况循环百公里油耗:

工况	油耗
工况 I（等速 25km/h，7.2s）	6.02 mL
工况 II（25→40km/h 加速，16.7s）	24.26 mL
工况 III（等速 40km/h，22.5s）	31.07 mL
工况 IV（40→50km/h 加速，14.0s）	23.24 mL
工况 V（等速 50km/h，18.0s）	32.25 mL
工况 VI（50→25km/h 减速，19.3s）	24.87 mL
总油耗 / 总行程	141.70 mL / 1075 m
六工况百公里油耗	13.18 L/100km

题目三：中型货车制动性能计算（习题 4.3）

3.1 题目描述

一中型货车装有前后制动器分开的双管路制动系，有关参数如下表所示：

载荷	质量(kg)	质心高 hg/m	轴距 L/m	质心至前轴 a/m	制动力分配 系数 β
空载	4080	0.845	3.950	2.100	0.38
满载	9290	1.170	3.950	2.950	0.38

- (1) 计算并绘制利用附着系数曲线和制动效率曲线。
- (2) 求行驶车速 $U_a=30\text{km/h}$ ，在 $\phi=0.80$ 路面上车轮不抱死的制动距离。
- (3) 求制动系前部/后部管路损坏时的制动距离。

3.2 计算公式

- (1) 利用附着系数：

$$\varphi_f = \beta \times z \times L / (b + z \times hg)$$

$$\varphi_r = (1-\beta) \times z \times L / (a - z \times hg)$$

- (2) 制动效率：

$$E_f = z / \varphi_f, E_r = z / \varphi_r$$

- (3) 同步附着系数：

$$\varphi_0 = (L \times \beta - b) / hg$$

- (4) 制动距离：

$$s = (\tau_1 + \tau_2/2) \times u_{a0} / 3.6 + u_{a0}^2 / (25.92 \times a_{bmax})$$

- (5) 最大制动减速度：

$$a_{bmax} = E \times \varphi \times g \text{ [m/s}^2\text{]}$$

式中， z 为制动强度， $b = L - a$ 为质心至后轴距离， $\tau_1=0.02\text{s}$ （制动系反应时间）， $\tau_2=0.02\text{s}$ （减速度上升时间）。

3.3 计算结果

- (1) 利用附着系数曲线和制动效率曲线如图 5 所示。

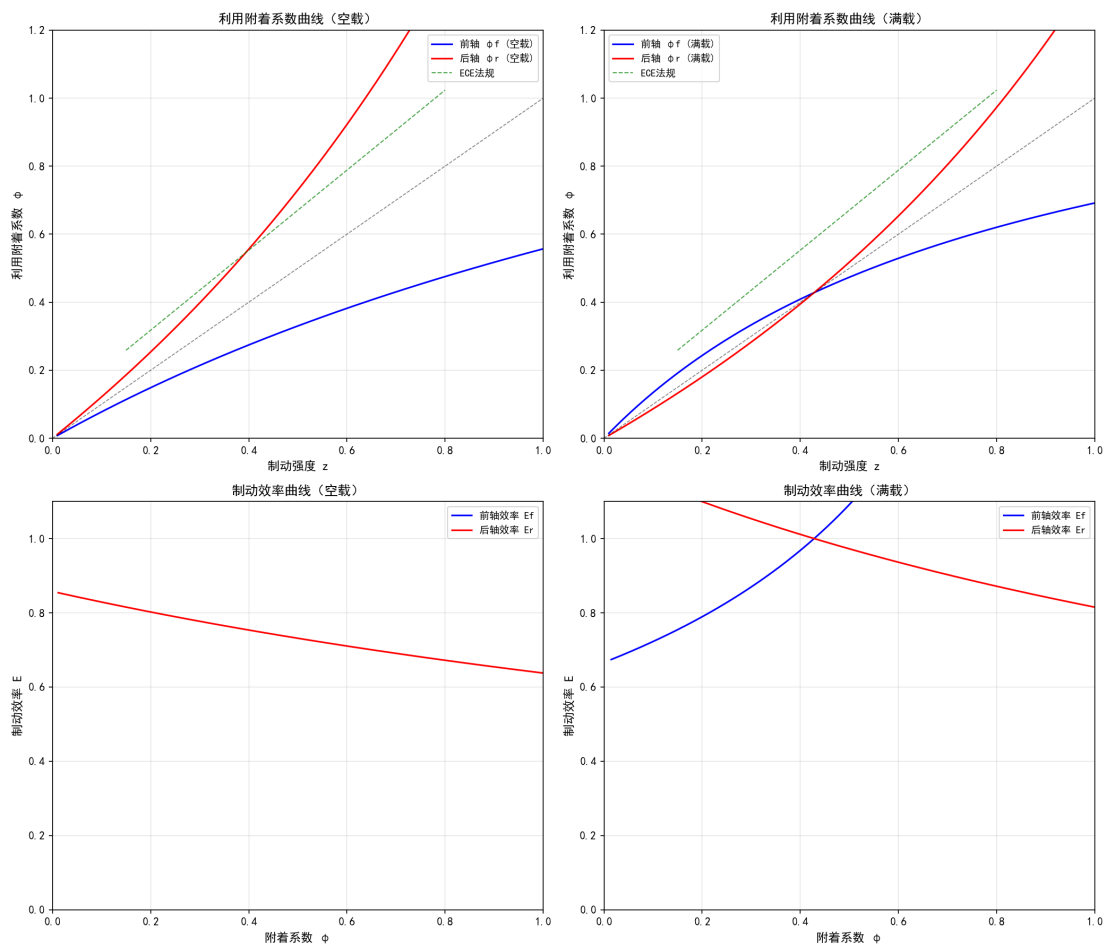


图 5 利用附着系数曲线与制动效率曲线

(2) 同步附着系数：

载荷状态	同步附着系数 ϕ_0
空载	-0.413
满载	0.428

(3) 制动距离 ($U_a=30\text{km/h}$, $\phi=0.80$)：

工况	制动效率 E	最大减速度	制动距离 s
满载正常制动	0.8715	6.83 m/s^2	5.33 m
空载正常制动	0.6720	5.27 m/s^2	6.84 m
前部管路损坏（仅后轮）	0.6038	4.73 m/s^2	7.59 m
后部管路损坏（仅前轮）	0.3318	2.60 m/s^2	13.60 m

附录：编程程序代码

A.1 题目一：动力性能计算程序

文件名: part1_dynamic_performance.py 运行环境: conda run -n modeling python part1_dynamic_performance.py

```
"""
汽车理论课程设计一（习题 1.3）
轻型货车动力性能计算 — 五档变速器
"""

import numpy as np
from scipy import integrate, interpolate
import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

# ===== 已知参数 =====
m_total = 3880      # 总质量 (kg)
m_load = 2000       # 装载质量 (kg)
m_curb = 1800       # 整备质量 (kg)
r_wheel = 0.367     # 车轮半径 (m)
eta_t = 0.85        # 传动系机械效率
f = 0.013           # 滚动阻力系数
CD_A = 2.77         # 空气阻力系数×迎风面积 (m²)
i0 = 5.83           # 主减速器传动比
If = 0.218          # 飞轮转动惯量 (kg·m²)
Iw1 = 1.798         # 二前轮转动惯量 (kg·m²)
Iw2 = 3.598         # 四后轮转动惯量 (kg·m²)
Iw_total = Iw1 + Iw2 # 总车轮转动惯量
L = 3.2             # 轴距 (m)
a = 1.947           # 质心至前轴距离（满载）(m)
hg = 0.9            # 质心高（满载）(m)
g = 9.8             # 重力加速度

# 五档变速器传动比
ig_list = np.array([5.56, 2.769, 1.644, 1.00, 0.793])
gear_labels = ['1 档', '2 档', '3 档', '4 档', '5 档']

n_min, n_max = 600, 4000 # 发动机转速范围 (r/min)
n = np.linspace(n_min, n_max, 500)

def engine_torque(n):
    """发动机外特性转矩 Tq (N·m), n in r/min"""
    x = n / 1000.0
    return -19.313 + 295.27 * x - 165.44 * x**2 + 40.874 * x**3 - 3.8445 * x**4
```

```

def vehicle_speed(n, ig):
    """车速 ua (km/h)"""
    return 0.377 * r_wheel * n / (ig * i0)

def driving_force(n, ig):
    """驱动力 Ft (N)"""
    Tq = engine_torque(n)
    return Tq * ig * i0 * eta_t / r_wheel

def rolling_resistance():
    """滚动阻力 Ff (N)"""
    return m_total * g * f

def air_resistance(ua):
    """空气阻力 Fw (N), ua in km/h"""
    return CD_A * ua**2 / 21.15

def rotating_mass_factor(ig):
    """旋转质量换算系数  $\delta$ """
    Iw_sum = Iw_total
    delta = 1 + (Iw_sum + If * ig**2 * i0**2 * eta_t) / (m_total * r_wheel**2)
    return delta

# ===== 1. 绘制驱动力-行驶阻力平衡图 =====
print("=" * 50)
print("题目一：轻型货车动力性能计算")
print("=" * 50)

plt.figure(figsize=(10, 7))

# 绘制各档驱动力曲线
max_ua = 0
for idx, ig in enumerate(ig_list):
    ua = vehicle_speed(n, ig)
    Ft = driving_force(n, ig)
    plt.plot(ua, Ft / 1000.0, label=f'Ft {gear_labels[idx]} (ig={ig})')
    max_ua = max(max_ua, ua[-1])

# 绘制阻力曲线
ua_range = np.linspace(0, max_ua * 1.05, 500)
Ff = np.full_like(ua_range, rolling_resistance())
Fw = air_resistance(ua_range)
F_total = (Ff + Fw) / 1000.0

```

```

Ff_plot = Ff / 1000.0

plt.plot(ua_range, Ff_plot, 'k--', label='滚动阻力 Ff', linewidth=1.5)
plt.plot(ua_range, (Ff + Fw) / 1000.0, 'k-', label='总阻力 Ff+Fw', linewidth=1.5)

plt.xlabel('车速 ua (km/h)', fontsize=12)
plt.ylabel('力 (kN)', fontsize=12)
plt.title('汽车驱动力-行驶阻力平衡图（五档变速器）', fontsize=14)
plt.legend(fontsize=9, loc='upper left')
plt.grid(True, alpha=0.3)
plt.xlim(0, ua_range[-1])
plt.ylim(0, None)
plt.tight_layout()
plt.savefig('fig1_driving_resistance_balance.png', dpi=150)
plt.close()
print("图1已保存: fig1_driving_resistance_balance.png")

# ===== 2. 求最高车速 =====
# 最高车速发生在最高档（5档）驱动力与总阻力曲线的交点
ig5 = ig_list[-1]
ua5 = vehicle_speed(n, ig5)
Ft5 = driving_force(n, ig5)
F_resist = rolling_resistance() + air_resistance(ua5)

# 寻找交点（驱动力 >= 阻力的最大车速）
diff = Ft5 - F_resist
# 找到从正到负的转折点
cross_idx = None
for i in range(len(diff) - 1):
    if diff[i] >= 0 and diff[i + 1] < 0:
        cross_idx = i
        break

if cross_idx is not None:
    # 线性插值精确求交点
    ua_max = ua5[cross_idx] + (ua5[cross_idx + 1] - ua5[cross_idx]) * \
        (0 - diff[cross_idx]) / (diff[cross_idx + 1] - diff[cross_idx])
else:
    ua_max = ua5[-1]

print(f"\n最高车速 ua_max = {ua_max:.2f} km/h")

# 最大转矩点
Tq_max = engine_torque(n)
Tq_max_val = np.max(Tq_max)
n_Tq_max = n[np.argmax(Tq_max)]
print(f"发动机最大转矩 Tq_max = {Tq_max_val:.2f} N·m @ n = {n_Tq_max:.0f} r/min")

# 最大功率点

```

```

Pe = engine_torque(n) * n / 9550.0
Pe_max = np.max(Pe)
n_Pe_max = n[np.argmax(Pe)]
print(f"发动机最大功率 Pe_max = {Pe_max:.2f} kW @ n = {n_Pe_max:.0f} r/min")

# ===== 3. 求最大爬坡度 =====
# 使用 1 档计算最大爬坡度
ig1 = ig_list[0]
ua1 = vehicle_speed(n, ig1)
Ft1 = driving_force(n, ig1)
Ff_val = rolling_resistance()

# 在 1 档最低稳定车速处计算最大爬坡度（通常在最大转矩点附近的车速）
# 取 1 档发动机最低转速以上范围
for i, ua_val in enumerate(ua1):
    if ua_val > 10: # 从 10km/h 以上开始（绕过很低车速不稳定的区域）
        idx_start = i
        break
else:
    idx_start = 0

# 取最大转矩对应的转速附近
idx_tq_max = np.argmax(engine_torque(n))
ua_at_max_Tq_1st = ua1[idx_tq_max]

# 计算各车速下的爬坡度
Ft1_sub = Ft1[idx_start:]
ua1_sub = ua1[idx_start:]
Fw1_sub = air_resistance(ua1_sub)

sin_alpha = (Ft1_sub - Ff_val - Fw1_sub) / (m_total * g)
sin_alpha = np.clip(sin_alpha, -1.0, 1.0)
alpha = np.arcsin(sin_alpha)
grade = np.tan(alpha) # i = tan(alpha)

max_grade_idx = np.argmax(grade)
max_grade = grade[max_grade_idx]
max_grade_ua = ua1_sub[max_grade_idx]
max_grade_alpha = np.degrees(alpha[max_grade_idx])

print(f"\n最大爬坡度（1 档）：")
print(f" 最大爬坡度 i = {max_grade:.4f} = {max_grade * 100:.2f}%")
print(f" 爬坡角度 α = {max_grade_alpha:.2f}°")
print(f" 对应车速 ua = {max_grade_ua:.2f} km/h")

# ===== 4. 计算最大爬坡度时的附着率 =====
# 后轮驱动货车，附着率  $C\phi_2 = F_{x2} / F_{z2}$ 
#  $F_{x2} = F_t - F_f - F_w$ （后轮驱动力）
#  $F_{z2} = mg \cdot a/L \cdot \cos(\alpha) + (F_f + F_w) \cdot hg/L + m \cdot g \cdot hg \cdot \sin(\alpha)/L \dots$ 

```

```

# 在最大爬坡度时
alpha_max = alpha[max_grade_idx]
Ft_at_max_grade = Ft1_sub[max_grade_idx]
Fw_at_max_grade = Fw1_sub[max_grade_idx]

# 后轴法向反力
Fz2 = (m_total * g * a * np.cos(alpha_max) +
        (Ft_at_max_grade - Ff_val - Fw_at_max_grade) * hg +
        m_total * g * hg * np.sin(alpha_max)) / L
# 后轮驱动力
Fx2 = Ft_at_max_grade - Ff_val - Fw_at_max_grade
C_phi2 = Fx2 / Fz2

print(f"\n 最大爬坡度时的附着率: ")
print(f"   后轴法向反力 Fz2 = {Fz2:.2f} N")
print(f"   后轮驱动力 Fx2 = {Fx2:.2f} N")
print(f"   附着率 Cφ2 = {C_phi2:.4f}")

# 不出现滑转的条件: Cφ2 < φ (附着系数)
# 一般干燥良好路面 φ≈0.7-0.8

# ===== 5. 加速度倒数曲线和加速时间 =====
# 2 档起步加速至 70 km/h
ig2 = ig_list[1]
ua2 = vehicle_speed(n, ig2)
Ft2 = driving_force(n, ig2)
delta2 = rotating_mass_factor(ig2)

# 加速度计算
Ff_val = rolling_resistance()
Fw2 = air_resistance(ua2)
a2 = (Ft2 - Ff_val - Fw2) / (delta2 * m_total)

# 只取加速度为正的部分, 且 2 档起步 (最低稳定车速以上)
valid_mask = (a2 > 0.001) & (ua2 >= ua2[0])
ua2_valid = ua2[valid_mask]
a2_valid = a2[valid_mask]

# 加速度倒数
inv_a2 = 1.0 / a2_valid

# 绘制加速度倒数曲线
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# 各档加速度曲线
ax1.set_title('各档加速度曲线', fontsize=12)
for idx, ig in enumerate(ig_list):
    ua_g = vehicle_speed(n, ig)

```

```

    Ft_g = driving_force(n, ig)
    Fw_g = air_resistance(ua_g)
    delta_g = rotating_mass_factor(ig)
    a_g = (Ft_g - Ff_val - Fw_g) / (delta_g * m_total)
    a_g = np.clip(a_g, 0, None)
    ax1.plot(ua_g, a_g, label=f'{gear_labels[idx]}')

ax1.set_xlabel('车速 ua (km/h)', fontsize=11)
ax1.set_ylabel('加速度 a (m/s²)', fontsize=11)
ax1.legend(fontsize=9)
ax1.grid(True, alpha=0.3)
ax1.set_xlim(0, ua_range[-1])

# 加速度倒数曲线（2 档）
ax2.set_title('2 档加速度倒数曲线', fontsize=12)
ax2.plot(ua2_valid, inv_a2, 'b-', linewidth=1.5)
ax2.set_xlabel('车速 ua (km/h)', fontsize=11)
ax2.set_ylabel('1/a (s²/m)', fontsize=11)
ax2.grid(True, alpha=0.3)

# 图解积分：计算从起步加速到 70km/h 的时间
target_speed = 70.0 # km/h
mask_70 = ua2_valid <= target_speed
ua_accel = ua2_valid[mask_70]
inv_a_accel = inv_a2[mask_70]

# 数值积分求加速时间（将速度单位转换为 m/s 再积分）
#  $t = \int (1/a) du$ , 其中  $u$  in m/s
#  $dt = (1/a) * du$ ,  $du$  in m/s
#  $ua$  in km/h  $\rightarrow u$  in m/s:  $u = ua/3.6$ 
#  $t = \int (1/a) * (1/3.6) du_a$  [km/h]
t_accel = integrate.trapezoid(inv_a_accel, ua_accel) / 3.6

# 填充积分区域
ax2.fill_between(ua_accel, 0, inv_a_accel, alpha=0.3, color='blue')
ax2.axvline(x=target_speed, color='r', linestyle='--', label=f'目标车速={target_speed} km/h')
ax2.legend(fontsize=9)
ax2.set_xlim(ua2_valid[0], ua2_valid[-1])
ax2.set_ylim(0, None)

plt.tight_layout()
plt.savefig('fig2_acceleration_curves.png', dpi=150)
plt.close()

# 打印加速时间
# 找到 2 档加速到 70km/h 过程中是否需要换挡
# 2 档车速范围
ua2_min = ua2[0]
ua2_max = ua2[-1]

```



```

print(f"\n2 档车速范围: {ua2_min:.1f} ~ {ua2_max:.1f} km/h")

# 检查 2 档是否能覆盖 0-70km/h
# 如果 2 档最高车速 < 70, 需要换 3 档
if ua2_max < target_speed:
    # 需要先在 2 档加速到 2 档最高车速, 然后换 3 档
    print("2 档无法直接达到 70km/h, 需要换 3 档继续加速")

    # 2 档加速到换挡车速
    mask_2nd = ua2_valid <= ua2_max
    ua_2nd_seg = ua2_valid[mask_2nd]
    inv_a_2nd_seg = inv_a2[mask_2nd]
    t_2nd = integrate.trapezoid(inv_a_2nd_seg, ua_2nd_seg) / 3.6

    # 3 档从相应车速继续加速
    ig3 = ig_list[2]
    ua3 = vehicle_speed(n, ig3)
    Ft3 = driving_force(n, ig3)
    Fw3 = air_resistance(ua3)
    delta3 = rotating_mass_factor(ig3)
    a3 = (Ft3 - Ff_val - Fw3) / (delta3 * m_total)

    valid_mask3 = (a3 > 0.001) & (ua3 <= target_speed) & (ua3 >= ua2_max)
    ua3_valid = ua3[valid_mask3]
    inv_a3 = 1.0 / a3[valid_mask3]
    t_3rd = integrate.trapezoid(inv_a3, ua3_valid) / 3.6

    # 加上换挡时间 (通常 0.5-1s)
    t_shift = 0.8
    t_total = t_2nd + t_3rd + t_shift

    print(f" 2 档加速段时间: {t_2nd:.2f} s")
    print(f" 换挡时间: {t_shift:.2f} s")
    print(f" 3 档加速段时间: {t_3rd:.2f} s")
    print(f" 2 档起步加速至 70km/h 总时间: {t_total:.2f} s")
else:
    print(f"\n2 档起步加速至{target_speed} km/h 的时间: ")
    print(f" 图解积分法 (梯形法则): t = {t_accel:.2f} s")

    # 也用 Simpson 法则验证
    if len(ua_accel) >= 3:
        t_accel_simpson = integrate.simpson(inv_a_accel, ua_accel) / 3.6
        print(f" 图解积分法 (Simpson 法则): t = {t_accel_simpson:.2f} s")

print("\n" + "=" * 50)
print("题目一计算完成!")
print("=" * 50)

```

A.2 题目二：燃油经济性计算程序

文件名: part2_fuel_economy.py 运行环境: conda run -n modeling
python part2_fuel_economy.py

```
"""
汽车理论课程设计二（习题 2.7）
轻型货车燃油经济性计算 — 功率平衡图 + 等速百公里油耗 + 六工况循环
"""

import numpy as np
from scipy import interpolate
import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

# ===== 已知参数（与题目一相同） =====
m_total = 3880
r_wheel = 0.367
eta_t = 0.85
f = 0.013
CD_A = 2.77
i0 = 5.83
g = 9.8

# 五档变速器
ig_list = np.array([5.56, 2.769, 1.644, 1.00, 0.793])
gear_labels = ['1 档', '2 档', '3 档', '4 档', '5 档']

n_min, n_max = 600, 4000
n = np.linspace(n_min, n_max, 500)

# 汽油参数 (93#)
rho_gasoline = 0.715    # 密度 (kg/L)
gamma_gasoline = rho_gasoline * g    # 重度 (N/L)

def engine_torque(n):
    x = n / 1000.0
    return -19.313 + 295.27 * x - 165.44 * x**2 + 40.874 * x**3 - 3.8445 * x**4

def engine_power(n):
    """发动机净功率 Pe (kW)"""
    Tq = engine_torque(n)
    return Tq * n / 9550.0

def vehicle_speed(n, ig):
    return 0.377 * r_wheel * n / (ig * i0)
```

```

def engine_speed_from_ua(ua, ig):
    """由车速反算发动机转速"""
    return ua * ig * i0 / (0.377 * r_wheel)

def rolling_resistance():
    return m_total * g * f

def air_resistance(ua):
    return CD_A * ua**2 / 21.15

def resistance_power(ua):
    """阻力功率 (kW)"""
    Ff = rolling_resistance()
    Fw = air_resistance(ua)
    return (Ff + Fw) * ua / (3600.0 * eta_t)

# ===== 负荷特性数据 =====
# 转速 (r/min) 和对应的拟合系数 B0~B4
load_data = {
    815: [1326.8, -416.46, 72.379, -5.8629, 0.17768],
    1207: [1354.7, -303.98, 36.657, -2.0553, 0.043072],
    1614: [1284.4, -189.75, 14.524, -0.51184, 0.0068164],
    2012: [1122.9, -121.59, 7.0035, -0.18517, 0.0018555],
    2603: [1141.0, -98.893, 4.4763, -0.091077, 0.00068906],
    3006: [1051.2, -73.714, 2.8593, -0.05138, 0.00035032],
    3403: [1233.9, -84.478, 2.9788, -0.047449, 0.00028230],
    3804: [1129.7, -45.291, 0.71113, -0.00075215, -0.000038568],
}

n_list = np.array(sorted(load_data.keys()))

def fuel_consumption_rate(Pe, n_target):
    """
    燃油消耗率 b [g/(kW·h)]
    Pe: 发动机功率 (kW)
    n_target: 发动机转速 (r/min)
    用相邻转速曲线插值法求得
    """
    # 找到相邻的两条转速曲线
    if n_target <= n_list[0]:
        n_low, n_high = n_list[0], n_list[1]
    elif n_target >= n_list[-1]:

```

```

        n_low, n_high = n_list[-2], n_list[-1]
    else:
        idx = np.searchsorted(n_list, n_target) - 1
        n_low, n_high = n_list[idx], n_list[idx + 1]

    # 在两条转速曲线上分别计算 b 值
    b_low = np.polyval(load_data[n_low][::-1], Pe)
    b_high = np.polyval(load_data[n_high][::-1], Pe)

    # 线性插值
    if n_high == n_low:
        return b_low
    b = b_low + (b_high - b_low) * (n_target - n_low) / (n_high - n_low)
    return b

# ===== 1. 功率平衡图 =====
print("=" * 50)
print("题目二：轻型货车燃油经济性计算")
print("=" * 50)

plt.figure(figsize=(10, 7))

# 发动机功率曲线
Pe_engine = engine_power(n)
plt.plot(n, Pe_engine, 'r-', linewidth=2, label='发动机功率 Pe')

# 绘制各档等速行驶时发动机功率（即阻力功率换算到发动机端）
ua_range = np.linspace(10, 120, 500)
# 阻力功率曲线（换算到发动机转速）
n_range = np.linspace(n_min, n_max, 500)
Pe_resist = resistance_power(vehicle_speed(n_range, ig_list[-1])) # 5 档阻力功率

# 各档阻力功率 vs 车速
for idx, ig in enumerate(ig_list):
    ua_g = vehicle_speed(n, ig)
    Pe_r = resistance_power(ua_g)
    plt.plot(n, Pe_r, '--', label=f'{gear_labels[idx]}阻力功率', alpha=0.7)

# 发动机外特性功率
plt.plot(n, Pe_engine, 'r-', linewidth=2.5, label='发动机功率 Pe')

plt.xlabel('发动机转速 n (r/min)', fontsize=12)
plt.ylabel('功率 (kW)', fontsize=12)
plt.title('汽车功率平衡图', fontsize=14)
plt.legend(fontsize=8, loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('fig3_power_balance.png', dpi=150)

```

```

plt.close()
print("图 3 已保存: fig3_power_balance.png")

# ===== 2. 等速百公里油耗曲线（最高档和次高档） =====
# 最高档 = 5 档 (ig=0.793), 次高档 = 4 档 (ig=1.00)

plt.figure(figsize=(9, 6))

for gear_idx, gear_name in [(4, '5 档（最高档）'), (3, '4 档（次高档）')]:
    ig = ig_list[gear_idx]
    # 计算该档位下不同车速对应的等速油耗
    ua_list_gear = np.linspace(20, vehicle_speed(n_max, ig), 100)

    Qs_list = []
    for ua_val in ua_list_gear:
        n_e = engine_speed_from_ua(ua_val, ig)
        if n_e < n_min or n_e > n_max:
            Qs_list.append(np.nan)
            continue

        Pe_req = resistance_power(ua_val)
        Pe_max_avail = engine_power(n_e) # 发动机在当前转速下的最大功率
        if Pe_req > Pe_max_avail * 0.95: # 需求功率超过可用功率
            Qs_list.append(np.nan)
            continue

        b_val = fuel_consumption_rate(Pe_req, n_e)
        Qs = Pe_req * b_val / (1.02 * ua_val * gamma_gasoline)
        Qs_list.append(Qs)

    Qs_list = np.array(Qs_list)
    valid = ~np.isnan(Qs_list)
    if valid.any():
        plt.plot(ua_list_gear[valid], Qs_list[valid], linewidth=2, label=gear_name)

plt.xlabel('车速 ua (km/h)', fontsize=12)
plt.ylabel('等速百公里油耗 Qs (L/100km)', fontsize=12)
plt.title('最高档与次高档等速百公里油耗曲线', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('fig4_constant_speed_fuel.png', dpi=150)
plt.close()
print("图 4 已保存: fig4_constant_speed_fuel.png")

# 打印一些特征值
for gear_idx, gear_name in [(4, '5 档（最高档）'), (3, '4 档（次高档）')]:
    ig = ig_list[gear_idx]
    ua_list_gear = np.linspace(30, vehicle_speed(n_max, ig), 200)

```

```

Qs_min = float('inf')
ua_best = 0
Qs_60 = None
Qs_80 = None

for ua_val in ua_list_gear:
    n_e = engine_speed_from_ua(ua_val, ig)
    if n_e < n_min or n_e > n_max:
        continue
    Pe_req = resistance_power(ua_val)
    if Pe_req > engine_power(n_e) * 0.95:
        continue
    b_val = fuel_consumption_rate(Pe_req, n_e)
    Qs = Pe_req * b_val / (1.02 * ua_val * gamma_gasoline)

    if Qs < Qs_min:
        Qs_min = Qs
        ua_best = ua_val
    if abs(ua_val - 60) < 0.5:
        Qs_60 = Qs
    if abs(ua_val - 80) < 0.5:
        Qs_80 = Qs

print(f"\n{gear_name}: ")
print(f" 最低油耗 {Qs_min:.2f} L/100km @ {ua_best:.0f} km/h")
if Qs_60:
    print(f" 60km/h 等速油耗: {Qs_60:.2f} L/100km")
if Qs_80:
    print(f" 80km/h 等速油耗: {Qs_80:.2f} L/100km")

# ===== 3. 六工况循环百公里油耗 =====
print(f"\n{' '*30}")
print("六工况循环百公里油耗计算")
print(f"{' '*30}")

# 六工况参数
# 工况, 累计行程(m), 时间(s), 累计时间(s), 车速范围(km/h), 说明
cycle = [
    {"name": "I", "dist": 50, "time": 7.2, "v_start": 25, "v_end": 25, "accel": 0},
    {"name": "II", "dist": 150, "time": 16.7, "v_start": 25, "v_end": 40, "accel": 0.25},
    {"name": "III", "dist": 250, "time": 22.5, "v_start": 40, "v_end": 40, "accel": 0},
    {"name": "IV", "dist": 175, "time": 14.0, "v_start": 40, "v_end": 50, "accel": 0.2},
    {"name": "V", "dist": 250, "time": 18.0, "v_start": 50, "v_end": 50, "accel": 0},
    {"name": "VI", "dist": 200, "time": 19.3, "v_start": 50, "v_end": 25, "accel": -0.36},
]

total_distance = 1075.0 # m
total_fuel_ml = 0.0

```

```

print("\n逐工况计算: ")
for seg in cycle:
    t_seg = seg["time"]
    n_steps = max(int(t_seg * 10), 50) # 每个工况至少 50 步
    t_arr = np.linspace(0, t_seg, n_steps)
    dt = t_arr[1] - t_arr[0]

    fuel_seg = 0.0

    for t_val in t_arr:
        # 当前车速 (km/h)
        if seg["accel"] == 0:
            v_kmh = seg["v_start"]
        else:
            v_kmh = seg["v_start"] + seg["accel"] * t_val * 3.6 # m/s² → km/h

        if v_kmh <= 0:
            continue

        # 计算所需发动机功率
        Ff = rolling_resistance()
        Fw = air_resistance(v_kmh)
        P_resist = (Ff + Fw) * v_kmh / (3600.0 * eta_t) # kW

        # 换挡策略: 根据车速选择合适挡位
        # 使用最高挡 (5 挡) 进行等速工况, 加速工况可能需降挡
        if seg["accel"] > 0 and v_kmh < 35:
            ig_used = ig_list[2] # 3 挡加速
        elif seg["accel"] > 0 and v_kmh < 45:
            ig_used = ig_list[3] # 4 挡加速
        else:
            ig_used = ig_list[4] # 5 挡

        n_e = engine_speed_from_ua(v_kmh, ig_used)
        if n_e < n_min:
            n_e = n_min
        if n_e > n_max:
            n_e = n_max

        # 发动机需要提供的功率 (需考虑附件消耗, 简化按等速行驶功率)
        Pe_actual = P_resist

        b_val = fuel_consumption_rate(Pe_actual, n_e)
        # b 值不应低于发动机最低燃油消耗率 (约 250-300 g/(kW·h))
        if b_val < 200:
            b_val = 200

        # 单位时间燃油消耗量 Qt = Pe * b / (3600 * ρ) [mL/s]
        # Pe [kW] * b [g/(kW·h)] / 3600 = g/s → /ρ [g/mL] = mL/s

```

```

Qt = Pe_actual * b_val / (3600.0 * rho_gasoline) # mL/s

fuel_seg += Qt * dt # mL

total_fuel_ml += fuel_seg
print(f" 工况{seg['name']}: 车速 {seg['v_start']}→{seg['v_end']} km/h, "
      f"时间 {seg['time']:.1f}s, 油耗 {fuel_seg:.2f} mL")

# 百公里油耗
Qs_cycle = total_fuel_ml * 100.0 / total_distance # L/100km
total_fuel_L = total_fuel_ml / 1000.0

print(f"\n 六工况循环总计: ")
print(f" 总行程: {total_distance:.0f} m")
print(f" 总油耗: {total_fuel_ml:.2f} mL = {total_fuel_L:.4f} L")
print(f" 百公里油耗: {Qs_cycle:.2f} L/100km")

print("\n" + "=" * 50)
print("题目二计算完成!")
print("=" * 50)

```


A.3 题目三：制动性能计算程序

文件名: part3_braking_performance.py 运行环境: conda run -n modeling python part3_braking_performance.py

```
"""
汽车理论课程设计三（习题 4.3）
中型货车制动性能计算 — 利用附着系数、制动效率、制动距离
"""
```

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False
```

```
# ===== 已知参数 =====
g = 9.8
```

```
# 空载参数
m_empty = 4080
hg_empty = 0.845
L_empty = 3.950
a_empty = 2.100
beta = 0.38 # 制动力分配系数
```

```
# 满载参数
m_full = 9290
hg_full = 1.170
L_full = 3.950
a_full = 2.950
beta_full = 0.38
```

```
# 制动系参数
tau1 = 0.02 # 制动系反应时间 (s)
tau2 = 0.02 # 制动减速度上升时间 (s)
ua0 = 30.0 # 初始车速 (km/h)
phi_road = 0.80 # 路面附着系数
```

```
def calc_adhesion_curves(m, hg, L, a, beta_val):
    """
    计算利用附着系数曲线和制动效率曲线
    返回: z_array, phi_f, phi_r, E_f, E_r
    """
    b = L - a # 质心至后轴距离
    z = np.linspace(0.01, 1.0, 200) # 制动强度

    # 利用附着系数
    phi_f = beta_val * z * L / (b + z * hg)
    phi_r = (1 - beta_val) * z * L / (a - z * hg)
```

```

# 制动效率（避免除零）
with np.errstate(divide='ignore', invalid='ignore'):
    E_f = np.where(phi_f > 1e-10, z / phi_f, 0)
    E_r = np.where(phi_r > 1e-10, z / phi_r, 0)

return z, phi_f, phi_r, E_f, E_r

def braking_distance(ua_kmh, phi, E, tau1, tau2):
    """
    计算制动距离
    ua_kmh: 初始车速 (km/h)
    phi: 路面附着系数
    E: 制动效率
    """
    ua_ms = ua_kmh / 3.6 # m/s
    a_bmax = E * phi * g # 最大制动减速度 (m/s²)

    if a_bmax <= 0:
        return float('inf')

    #  $s = (\tau_1 + \tau_2/2) * ua_0 / 3.6 + ua_0^2 / (25.92 * a_{bmax})$ 
    s = (tau1 + tau2 / 2) * ua_kmh / 3.6 + ua_kmh**2 / (25.92 * a_bmax)
    return s

# ===== 1. 绘制利用附着系数曲线和制动效率曲线 =====
print("=" * 50)
print("题目三：中型货车制动性能计算")
print("=" * 50)

fig, axes = plt.subplots(2, 2, figsize=(14, 12))

# 空载
z_empty, phi_f_empty, phi_r_empty, E_f_empty, E_r_empty = \
    calc_adhesion_curves(m_empty, hg_empty, L_empty, a_empty, beta)

# 满载
z_full, phi_f_full, phi_r_full, E_f_full, E_r_full = \
    calc_adhesion_curves(m_full, hg_full, L_full, a_full, beta_full)

# 利用附着系数—空载
ax = axes[0, 0]
ax.plot(z_empty, phi_f_empty, 'b-', label='前轴  $\phi_f$  (空载)', linewidth=1.5)
ax.plot(z_empty, phi_r_empty, 'r-', label='后轴  $\phi_r$  (空载)', linewidth=1.5)
ax.plot([0, 1], [0, 1], 'k--', linewidth=0.8, alpha=0.5) #  $\phi=z$  参考线
# ECE 法规线
z_ECE = np.linspace(0.15, 0.8, 100)

```

```

phi_ECE = (z_ECE + 0.07) / 0.85
ax.plot(z_ECE, phi_ECE, 'g--', linewidth=1, alpha=0.7, label='ECE 法规')
ax.set_xlabel('制动强度 z', fontsize=11)
ax.set_ylabel('利用附着系数  $\phi$ ', fontsize=11)
ax.set_title('利用附着系数曲线（空载）', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 1.0)
ax.set_ylim(0, 1.2)

# 利用附着系数—满载
ax = axes[0, 1]
ax.plot(z_full, phi_f_full, 'b-', label='前轴  $\phi_f$  (满载)', linewidth=1.5)
ax.plot(z_full, phi_r_full, 'r-', label='后轴  $\phi_r$  (满载)', linewidth=1.5)
ax.plot([0, 1], [0, 1], 'k--', linewidth=0.8, alpha=0.5)
ax.plot(z_ECE, phi_ECE, 'g--', linewidth=1, alpha=0.7, label='ECE 法规')
ax.set_xlabel('制动强度 z', fontsize=11)
ax.set_ylabel('利用附着系数  $\phi$ ', fontsize=11)
ax.set_title('利用附着系数曲线（满载）', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 1.0)
ax.set_ylim(0, 1.2)

# 制动效率—空载
ax = axes[1, 0]
# 只绘制有效范围内的效率（附着系数不超过路面附着系数的典型值）
mask_f_empty = phi_f_empty <= 1.2
mask_r_empty = phi_r_empty <= 1.2
ax.plot(phi_f_empty[mask_f_empty], E_f_empty[mask_f_empty], 'b-', label='前轴效率 Ef',
        linewidth=1.5)
ax.plot(phi_r_empty[mask_r_empty], E_r_empty[mask_r_empty], 'r-', label='后轴效率 Er',
        linewidth=1.5)
ax.set_xlabel('附着系数  $\phi$ ', fontsize=11)
ax.set_ylabel('制动效率 E', fontsize=11)
ax.set_title('制动效率曲线（空载）', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 1.0)
ax.set_ylim(0, 1.1)

# 制动效率—满载
ax = axes[1, 1]
mask_f_full = phi_f_full <= 1.2
mask_r_full = phi_r_full <= 1.2
ax.plot(phi_f_full[mask_f_full], E_f_full[mask_f_full], 'b-', label='前轴效率 Ef',
        linewidth=1.5)
ax.plot(phi_r_full[mask_r_full], E_r_full[mask_r_full], 'r-', label='后轴效率 Er',
        linewidth=1.5)

```

```

ax.set_xlabel('附着系数  $\phi$ ', fontsize=11)
ax.set_ylabel('制动效率 E', fontsize=11)
ax.set_title('制动效率曲线（满载）', fontsize=12)
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 1.0)
ax.set_ylim(0, 1.1)

plt.tight_layout()
plt.savefig('fig5_adhesion_efficiency.png', dpi=150)
plt.close()
print("图 5 已保存: fig5_adhesion_efficiency.png")

# ===== 2. 制动距离计算（满载,  $U_a=30\text{km/h}$ ,  $\phi=0.80$ ） =====
print(f"\n--- 制动距离计算（{ua0}km/h,  $\phi=\{\text{phi\_road}\}$ ） ---")

# 满载时的制动效率
# 找到满载时在  $\phi=0.80$  路面上对应哪个轴的制动效率
# 前轴:  $\phi_f = \beta \cdot z \cdot L / (b + z \cdot h_g)$ , 当  $\phi_f=0.80$  时求 z
b_full = L_full - a_full # 质心至后轴距离

# 在  $\phi=0.80$  路面上, 前轴先抱死还是后轴先抱死取决于利用附着系数
# 求前轴利用附着系数=0.80 时的制动强度  $z_f$ 
#  $\beta \cdot z \cdot L / (b + z \cdot h_g) = 0.80$ 
#  $\beta \cdot z \cdot L = 0.80 \cdot (b + z \cdot h_g)$ 
#  $\beta \cdot z \cdot L - 0.80 \cdot z \cdot h_g = 0.80 \cdot b$ 
#  $z \cdot (\beta \cdot L - 0.80 \cdot h_g) = 0.80 \cdot b$ 
z_f_at_phi = (phi_road * b_full) / (beta * L_full - phi_road * hg_full)
# 后轴利用附着系数=0.80 时的制动强度  $z_r$ 
z_r_at_phi = (phi_road * a_full) / ((1 - beta) * L_full + phi_road * hg_full)

print(f" 前轴在  $\phi=0.80$  时对应的制动强度  $z_f = \{z\_f\_at\_phi:.4f\}$ ")
print(f" 后轴在  $\phi=0.80$  时对应的制动强度  $z_r = \{z\_r\_at\_phi:.4f\}$ ")

# 较小者先抱死, 决定制动效率
# 找前轴和后轴在  $\phi=0.80$  时的制动效率
# 插值法
E_f_at_phi = np.interp(phi_road, phi_f_full, E_f_full)
E_r_at_phi = np.interp(phi_road, phi_r_full, E_r_full)

print(f" 满载前轴制动效率  $E_f(\phi=0.80) = \{E\_f\_at\_phi:.4f\}$ ")
print(f" 满载后轴制动效率  $E_r(\phi=0.80) = \{E\_r\_at\_phi:.4f\}$ ")

# 制动效率取较小者（先抱死的轴决定整车制动效率）
E_effective = min(E_f_at_phi, E_r_at_phi)
print(f" 整车有效制动效率  $E = \{E\_effective:.4f\}$ ")

# 制动距离
s_normal = braking_distance(ua0, phi_road, E_effective, tau1, tau2)

```

```

a_bmax_normal = E_effective * phi_road * g

print(f"\n 正常制动距离: s = {s_normal:.2f} m")
print(f" 最大制动减速度: a_bmax = {a_bmax_normal:.2f} m/s²")

# 空载情况
E_f_empty_phi = np.interp(phi_road, phi_f_empty, E_f_empty)
E_r_empty_phi = np.interp(phi_road, phi_r_empty, E_r_empty)
E_effective_empty = min(E_f_empty_phi, E_r_empty_phi)
s_empty = braking_distance(ua0, phi_road, E_effective_empty, tau1, tau2)
a_bmax_empty = E_effective_empty * phi_road * g

print(f"\n 空载制动距离: s = {s_empty:.2f} m")
print(f" 空载最大制动减速度: a_bmax = {a_bmax_empty:.2f} m/s²")
print(f" 空载制动效率 E = {E_effective_empty:.4f}")

# ===== 3. 管路损坏时的制动距离 =====
print(f"\n--- 制动管路损坏时的制动距离 ---")

# 前部管路损坏: 只有后轮制动,  $\beta=0$ 
beta_rear_only = 0.0
z_r_fail, phi_f_r_fail, phi_r_r_fail, E_f_r_fail, E_r_r_fail = \
    calc_adhesion_curves(m_full, hg_full, L_full, a_full, beta_rear_only)

#  $\phi=0.80$  时后轮制动效率
phi_r_rear_fail_at_phi = np.interp(phi_road, phi_r_r_fail, E_r_r_fail) if \
    phi_r_r_fail[-1] >= phi_road else 0

# 对于前部管路损坏, 后轴利用附着系数=0.80 对应制动强度
#  $(1-0)*z*L/(a - z*hg) = z*L/(a - z*hg) = 0.80 \rightarrow z = 0.80*a/(L+0.80*hg)$ 
z_rear_only = (phi_road * a_full) / (L_full + phi_road * hg_full)
E_rear_only = z_rear_only / phi_road
s_rear_fail = braking_distance(ua0, phi_road, E_rear_only, tau1, tau2)
a_bmax_rear_fail = E_rear_only * phi_road * g

print(f"\n【前部管路损坏（仅后轮制动）】")
print(f" 制动强度 z = {z_rear_only:.4f}")
print(f" 制动效率 E = {E_rear_only:.4f}")
print(f" 制动距离 s = {s_rear_fail:.2f} m")
print(f" 最大制动减速度 a_bmax = {a_bmax_rear_fail:.2f} m/s²")

# 后部管路损坏: 只有前轮制动,  $\beta=1$ 
beta_front_only = 1.0
z_f_fail, phi_f_f_fail, phi_r_f_fail, E_f_f_fail, E_r_f_fail = \
    calc_adhesion_curves(m_full, hg_full, L_full, a_full, beta_front_only)

#  $\beta=1$  时, 前轴利用附着系数=0.80 对应制动强度
#  $1*z*L/(b+z*hg) = 0.80 \rightarrow z = 0.80*b/(L-0.80*hg)$ 
z_front_only = (phi_road * b_full) / (L_full - phi_road * hg_full)

```

```

E_front_only = z_front_only / phi_road
s_front_fail = braking_distance(ua0, phi_road, E_front_only, tau1, tau2)
a_bmax_front_fail = E_front_only * phi_road * g

print(f"\n【后部管路损坏（仅前轮制动）】")
print(f"  制动强度 z = {z_front_only:.4f}")
print(f"  制动效率 E = {E_front_only:.4f}")
print(f"  制动距离 s = {s_front_fail:.2f} m")
print(f"  最大制动减速度 a_bmax = {a_bmax_front_fail:.2f} m/s²")

# 汇总对比
print(f"\n{' '*55}")
print(f"制动距离汇总（{ua0}km/h,  $\phi$ = {phi_road}）:")
print(f"{' '*55}")
print(f"  满载正常制动:      {s_normal:.2f} m")
print(f"  空载正常制动:      {s_empty:.2f} m")
print(f"  前部管路损坏:      {s_rear_fail:.2f} m")
print(f"  后部管路损坏:      {s_front_fail:.2f} m")
print(f"{' '*55}")

# ===== 关键数据点输出 =====
print(f"\n--- 关键附着系数分析 ---")
# 空载同步附着系数
b_empty = L_empty - a_empty
phi0_empty = (L_empty * beta - b_empty) / hg_empty
print(f"空载同步附着系数  $\phi_0$  = {phi0_empty:.4f}")

# 满载同步附着系数
b_full = L_full - a_full
phi0_full = (L_full * beta_full - b_full) / hg_full
print(f"满载同步附着系数  $\phi_0$  = {phi0_full:.4f}")

# 分析制动稳定性
print(f"\n制动稳定性分析:")
for label, phi0, cond in [("空载", phi0_empty, "空载"), ("满载", phi0_full, "满载")]:
    if phi0 > phi_road:
        print(f"  {label}:  $\phi_0$ = {phi0:.2f} >  $\phi$ = {phi_road}, 前轮先抱死, 制动稳定")
    else:
        print(f"  {label}:  $\phi_0$ = {phi0:.2f} <  $\phi$ = {phi_road}, 后轮先抱死, 存在侧滑风险")

print("\n" + "=" * 50)
print("题目三计算完成!")
print("=" * 50)

```